Universidade Estadual de Campinas

Faculdade de Engenharia Elétrica e de Computação

Controlador AEB

Modelo de Simulação MIL (Model-in-the-Loop)

Implementação Fiel ao Código C Embarcado

Rodrigo Faguetti

Março de 2026

Arquivos: `AEB_Controller.slx` • `AEB_Controller_build.m` • `AEB_Controller_scenarios.m`

Sumário

1 Introdução **2**
1.1 Cenários de validação 2

2 Arquitetura do Controlador **2**
2.1 Janela de velocidade 3

3 Cálculo de TTC e Piso de Distância **3**
3.1 Fórmula do TTC 3
3.2 Piso de distância d_{brake} 3
3.3 Limiares de TTC 4

4 Máquina de Estados Finita (FSM) **5**
4.1 Descrição dos estados 5
4.2 Temporizadores e histerese 6
4.3 Pisos de distância 6
4.4 Código MATLAB Function (FSM) 6

5 Controlador PI com Limitação de Jerk **7**
5.1 Equações do controlador 8
5.2 Parâmetros (aeb_config.h) 8
5.3 Resultado da simulação 9

6 Lógica de Alertas **9**

7 Override do Motorista **10**

8 Modelo Simulink (AEB_Controller.slx) **11**
8.1 Geração programática 11
8.2 Sinais de entrada (From Workspace) 12
8.3 Sinais de saída (To Workspace) 12

9 Resultados de Simulação **12**
9.1 Comparação de cenários Euro NCAP 12
9.2 Efeito do piso de distância 14

10 Conformidade com Requisitos **15**

11 Conclusão **15**

1 Introdução

O sistema de *Frenagem Autônoma de Emergência* (AEB — *Autonomous Emergency Braking*) é um sistema de segurança ativa classificado como **ASIL-B** (Automotive Safety Integrity Level B) pela norma ISO 26262. Sua função é detectar a iminência de uma colisão frontal e aplicar automaticamente a frenagem para reduzir ou eliminar o impacto, caso o motorista não reaja a tempo.

Este relatório documenta a camada de controle do sistema AEB, implementada como um modelo **MIL (Model-in-the-Loop)** no ambiente MATLAB/Simulink R2025b. O modelo é uma transcrição fiel do código C embarcado contido nos arquivos:

- `aeb_fsm.c` — Máquina de estados finita de 7 estados;
- `aeb_pid.c` — Controlador PI com anti-windup e limitação de jerk;
- `aeb_ttc.c` — Cálculo do TTC e piso de distância d_{brake};
- `aeb_alert.c` — Geração de alertas visuais e sonoros.

O modelo Simulink (`AEB_Controller.slx`) é gerado programaticamente pelo script `AEB_Controller_build.m` e organizado em cinco subsistemas, cada um correspondendo diretamente a um arquivo C.

1.1 Cenários de validação

Os cenários implementados seguem o protocolo Euro NCAP para sistemas AEB [1]:

- **CCRs 50 km/h** — Colisão com obstáculo estático, 50 km/h;
- **CCRs 30 km/h** — Colisão com obstáculo estático, 30 km/h;
- **CCRm** — Alvo em movimento a 20 km/h, ego a 50 km/h;
- **Override** — Intervenção do motorista pelo pedal de freio;
- **Fault sensor** — Degradação de confiança da percepção;
- **AEB disabled** — Sistema desabilitado.

2 Arquitetura do Controlador

O controlador AEB é composto pelos subsistemas apresentados na Figura 1 e detalhados nas seções seguintes.

Entradas	Subsistema	Saídas
d, v_{ego}, v_{tgt}	`TTC_Calc`	TTC, d_{brake}, is_closing
δ_{steer}, brake_pedal	`Driver_Override`	override_active
TTC, d_{brake}, d, v_{ego}, conf, override	`AEB_FSM`	fsm_state, decel_target, brake_active
decel_target, a_{ego}	`PID_Brake`	brake_pct, brake_bar
fsm_state	`Alert_Logic`	alert_type, buzzer_cmd

Figura 1: Arquitetura em blocos do controlador AEB.

2.1 Janela de velocidade

O sistema AEB opera apenas dentro da janela de velocidade definida em `aeb_config.h`:

$$V_{\text{MIN}} = 1{,}39 \text{ m/s (5 km/h)} \quad \leq \quad v_{\text{ego}} \quad \leq \quad V_{\text{MAX}} = 16{,}67 \text{ m/s (60 km/h)}$$

Fora desta janela a FSM permanece em `STANDBY`. O sistema continua monitorando a velocidade mesmo durante a frenagem, pois quando o veículo para ($v < V_{\text{MIN}}$) a FSM transita para `POST_BRAKE`.

3 Cálculo de TTC e Piso de Distância

3.1 Fórmula do TTC

O Tempo para Colisão (*Time-To-Collision*, TTC) é calculado em `aeb_ttc.c` conforme:

$$\text{TTC} = \begin{cases} \min\left(\dfrac{d}{v_{\text{rel}}},\ \text{TTC}_{\text{MAX}}\right) & \text{se } v_{\text{rel}} > V_{\text{REL_MIN}} \\ \text{TTC}_{\text{MAX}} & \text{caso contrário} \end{cases} \tag{1}$$

onde $v_{\text{rel}} = v_{\text{ego}} - v_{\text{tgt}}$ é a velocidade relativa de aproximação, $V_{\text{REL_MIN}} = 0{,}5$ m/s evita divisão por zero e $\text{TTC}_{\text{MAX}} = 10{,}0$ s.

3.2 Piso de distância d_{brake}

Para garantir que o veículo consiga parar antes de atingir o obstáculo, o sistema calcula a distância mínima de parada:

$$d_{\text{brake}} = \frac{v_{\text{ego}}^2}{2 \cdot \text{DECEL_L3}} \tag{2}$$

com DECEL_L3 = 6,0 m/s^2. Se $d_{\text{brake}} \geq d$ (o veículo não consegue parar), a FSM ativa imediatamente `BRAKE_L3`, independentemente do valor do TTC. Este

mecanismo é denominado *piso de distância* (*braking floor*).

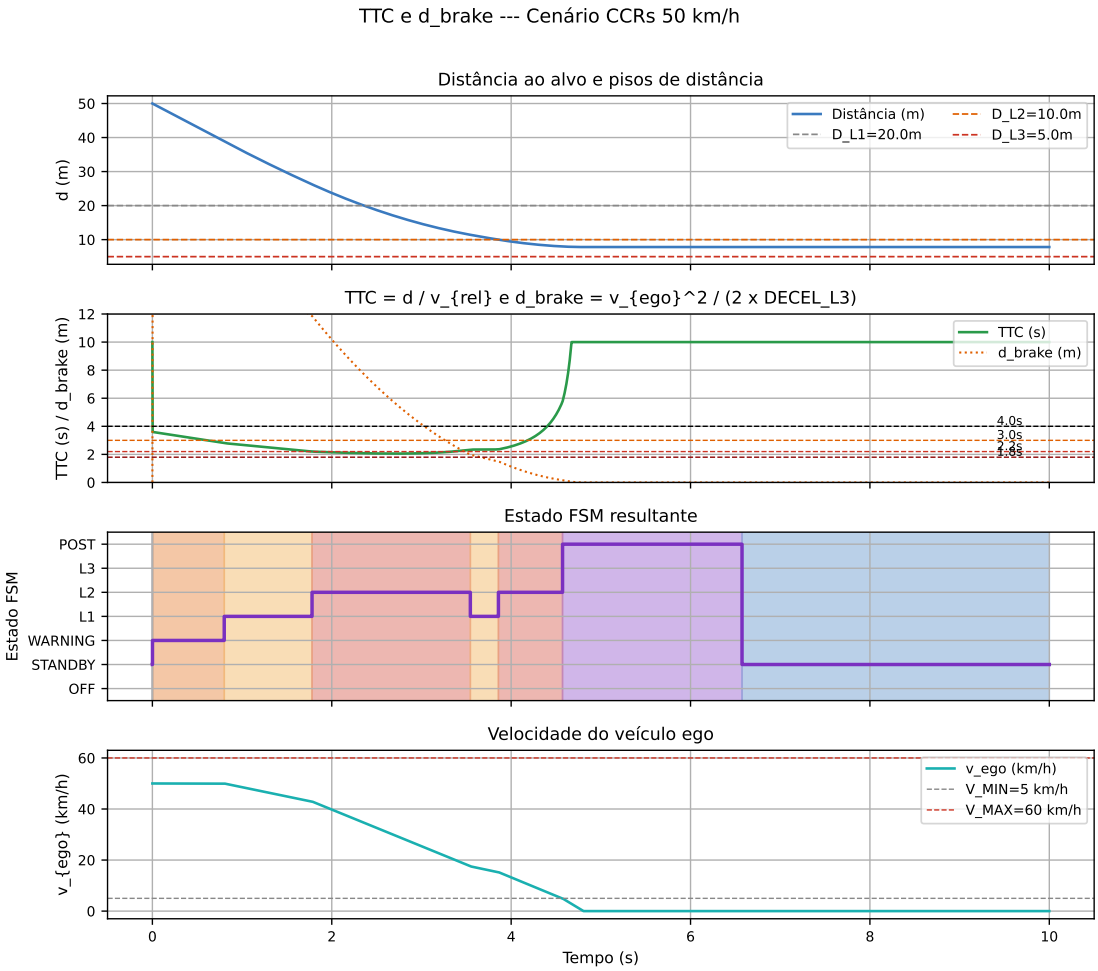

Figura 2: TTC calculado, d_{brake} e limiares de intervenção — cenário CCRs 50 km/h.

3.3 Limiares de TTC

Os limiares definem os níveis de intervenção:

Tabela 1: Limiares de TTC e níveis de frenagem.

TTC $\leq$	Nível	Desaceleração alvo	Pressão aprox.
4,0 s	WARNING	0,0 m/s^2	0 bar
3,0 s	BRAKE_L1	2,0 m/s^2	2 bar
2,2 s	BRAKE_L2	4,0 m/s^2	4 bar
1,8 s	BRAKE_L3	6,0 m/s^2	6 bar

4 Máquina de Estados Finita (FSM)

A FSM implementada em `aeb_fsm.c` gerencia 7 estados e é o núcleo de decisão do sistema AEB.

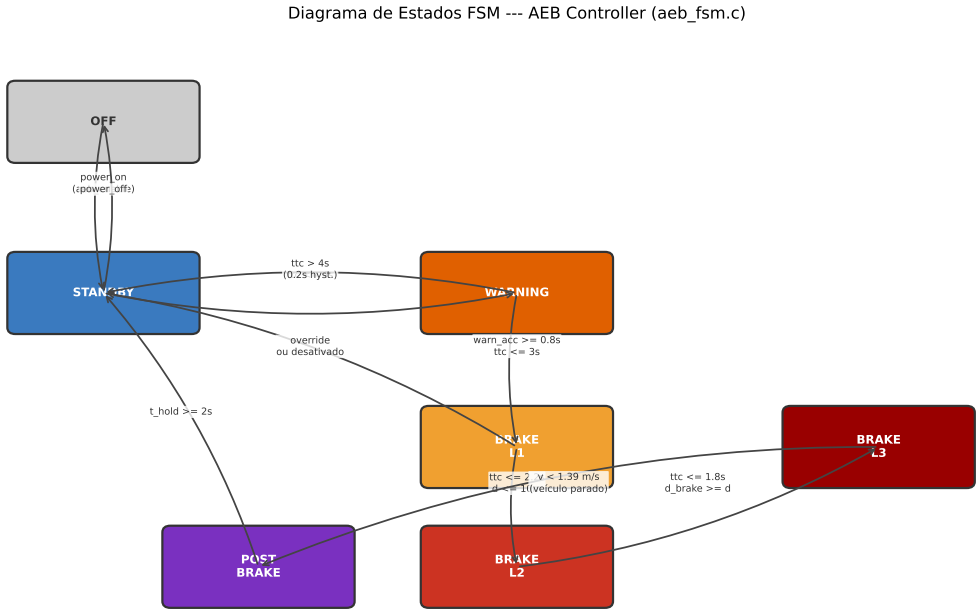

Figura 3: Diagrama de estados da FSM — implementação de `aeb_fsm.c`.

4.1 Descrição dos estados

`OFF (0)`

Sistema desligado. Transita para `STANDBY` com `aeb_enable = 1`.

`STANDBY (1)`

Sistema ativo, monitorando. Não há ameaça detectada ou veículo fora da janela de velocidade. Transita para `WARNING` quando TTC $\leq$ 4,0 s e a velocidade está na janela.

`WARNING (2)`

Alerta visual e sonoro (beep único). Acumula o timer `warn_acc`. Transita para `BRAKE_L1` apenas após `warn_acc` $\geq$ 0,8 s (requisito FR-AEB-007).

`BRAKE_L1 (3)`

Frenagem leve: desaceleração alvo de 2,0 m/s^2, ativada quando TTC $\leq$ 3,0 s ou $d \leq 20$ m.

`BRAKE_L2 (4)`

Frenagem moderada: 4,0 m/s^2, ativada quando TTC $\leq$ 2,2 s ou $d \leq 10$ m.

`BRAKE_L3 (5)`

Frenagem máxima: 6,0 m/s^2, ativada quando TTC $\leq$ 1,8 s, $d \leq 5$ m ou $d_{\text{brake}} \geq d$.

`POST_BRAKE (6)`

Mantém a desaceleração de L3 por 2,0 s após o veículo parar (requisito FR-BRK-005), impedindo repartida não intencional.

4.2 Temporizadores e histerese

Para evitar oscilações de estado, a FSM utiliza:

- **warn_acc**: Acumulador de tempo em `WARNING`. Deve atingir 0,8 s antes que a escalada para frenagem seja permitida.
- **deb_t**: Timer de histerese para desescalada. A redução de nível (p.ex. de L2 para L1) exige 0,2 s consecutivos no nível menor.
- **pb_t**: Timer de `POST_BRAKE`. Após 2,0 s, o sistema retorna a `STANDBY`.

4.3 Pisos de distância

Mesmo que o TTC melhore (obstáculo desacelerando ou virando), enquanto a distância estiver abaixo dos pisos, o nível mínimo de frenagem é mantido:

Tabela 2: Pisos de distância por nível de frenagem.

Estado	Piso	Razão
BRAKE_L1	$d \leq 20$ m	Proximidade moderada
BRAKE_L2	$d \leq 10$ m	Proximidade crítica
BRAKE_L3	$d \leq 5$ m	Proximidade extrema

4.4 Código MATLAB Function (FSM)

O bloco `MATLAB Function` dentro do subsistema `AEB_FSM` transcreve fielmente a lógica de `aeb_fsm.c`:

Listing 1: Núcleo da FSM — MATLAB Function (trecho).

```
persistent st st_t warn_acc deb_t pb_t;
if isempty(st), st=1; st_t=0; warn_acc=0; deb_t=0; pb_t=0; end
DT=0.001; HYST=0.2; WARN_MIN=0.8; POST=2.0;
DEC=[0,0,0,2.0,4.0,6.0,6.0];  % POST_BRAKE mantem L3
```

```
% Ameaca desejada (avalia TTC + piso d_brake)
if ttc <= TTC_L3 || d_brake >= distance
    des = 5;  % BRAKE_L3
elseif ttc <= TTC_L2 || distance <= D_L2
    des = 4;
elseif ttc <= TTC_L1 || distance <= D_L1
    des = 3;
elseif ttc <= TTC_W
    des = 2;
else
    des = 1;
end

% Transicoes com timers
switch st
  case 2  % WARNING
    warn_acc = warn_acc + DT;
    if warn_acc >= WARN_MIN && des >= 3
        st = des; st_t = 0;
    end
  case {3,4,5}  % BRAKE_Lx
    if des < st
        deb_t = deb_t + DT;
        if deb_t >= HYST
            if des < 2, st=6; pb_t=0; else st=des; end
            deb_t=0;
        end
    end
end
```

5 Controlador PI com Limitação de Jerk

O controlador de pressão de freio é implementado em `aeb_pid.c` como um controlador PI com:

- Anti-windup no integrador ([0, 50]%);
- Saturação da saída ([0, 100]%);
- Limitação de jerk máximo de $\dot{p}_{\max} = 1{,}0$ %/ciclo.

5.1 Equações do controlador

$$e(k) = a_{\text{target}}(k) - a_{\text{actual}}(k) \tag{3}$$

$$I(k) = \text{clamp}(I(k-1) + K_i \cdot e(k) \cdot \Delta t,\ 0,\ 50) \tag{4}$$

$$u_{\text{raw}}(k) = \text{clamp}(K_p \cdot e(k) + I(k),\ 0,\ 100) \tag{5}$$

$$\Delta u = \text{clamp}(u_{\text{raw}}(k) - u(k-1),\ -\delta_{\max},\ +\delta_{\max}) \tag{6}$$

$$\text{brake_pct}(k) = \text{clamp}(u(k-1) + \Delta u,\ 0,\ 100) \tag{7}$$

$$\text{brake_bar}(k) = \text{brake_pct}(k) \times 0{,}1 \tag{8}$$

com $K_p = 10{,}0$, $K_i = 0{,}05$, $\Delta t = 0{,}001$ s e $\delta_{\max} = 1{,}0$ %/ciclo.

5.2 Parâmetros (aeb_config.h)

Tabela 3: Parâmetros do controlador PI.

Parâmetro	Valor	Justificativa
K_p (PID_KP)	10,0	Resposta rápida a erros de desaceleração
K_i (PID_KI)	0,05	Eliminação de erro em regime permanente
MAX_JERK	100 m/s^3	ISO 15622: conforto do passageiro
INTEG_MAX	50%	Anti-windup: limita acúmulo integral
OUT_MAX	100%	Saturação física da válvula

5.3 Resultado da simulação

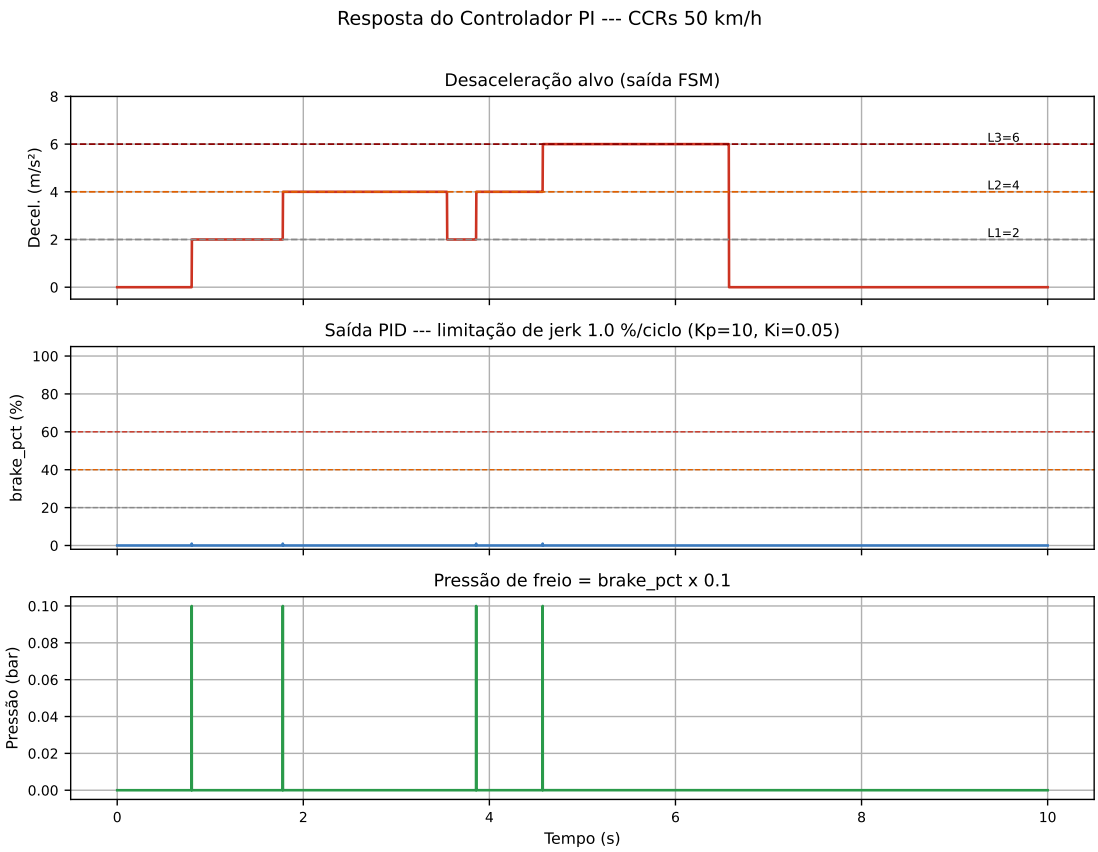

Figura 4: Resposta do controlador PI — desaceleração alvo, brake_pct e pressão de freio.

6 Lógica de Alertas

A geração de alertas é implementada em `aeb_alert.c`. Os alertas são transmitidos pela mensagem CAN `AEB_AlertCmd` (ID 0x300):

Tabela 4: Mapeamento estado FSM — alertas.

Estado	Nível	Visual	Sonoro	BuzzerCmd (0x300)
OFF/STANDBY	0	Off	Off	Off (0)
WARNING	1	On	On	SingleBeep (1)
BRAKE_L1	2	On	On	DoubleBeep (2)
BRAKE_L2	3	On	On	FastPulse (4)
BRAKE_L3	3	On	On	ContinuousBeep (3)
POST_BRAKE	1	On	Off	Off (0)

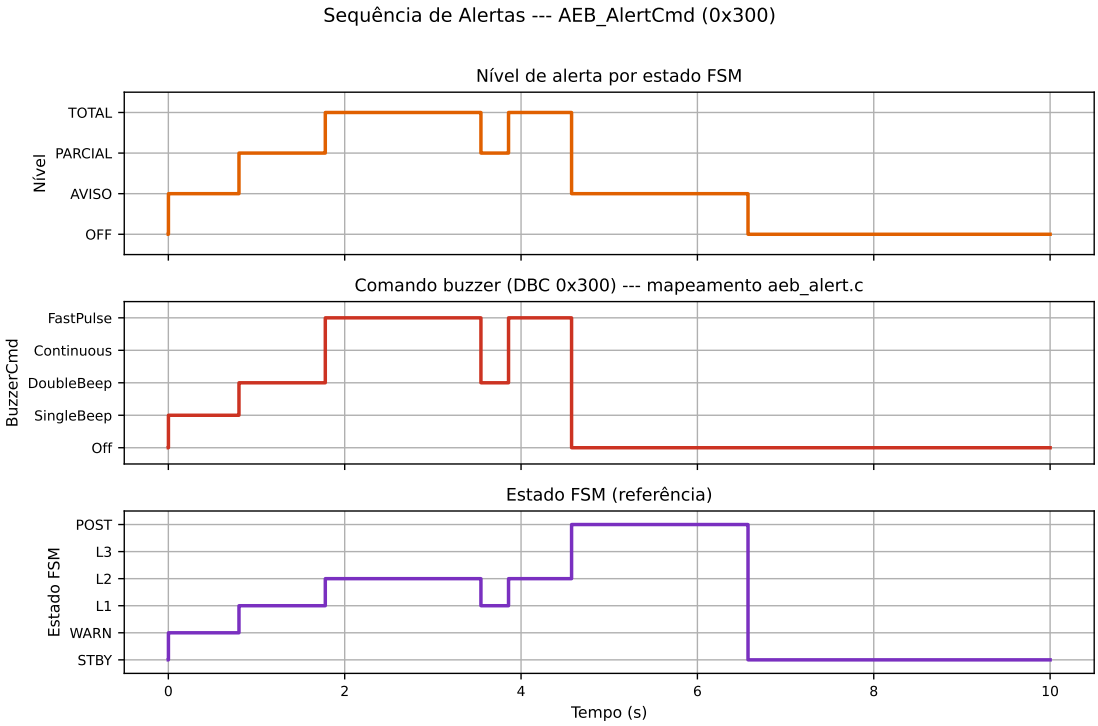

Figura 5: Sequência de alertas por estado FSM — cenário CCRs 50 km/h.

7 Override do Motorista

O subsistema `Driver_Override` implementa a função `driver_override()` de `aeb_fsm.c`. O override é ativo se:

$$\text{override} = (\text{brake_pedal} \neq 0) \;\vee\; (|\delta_{\text{steer}}| > 5\check{r}) \tag{9}$$

Quando o override é detectado, a FSM retorna imediatamente a `STANDBY`, anulando qualquer comando de frenagem em curso. Isso respeita o princípio de que o motorista tem sempre prioridade sobre o sistema automatizado (requisito FR-AEB-003).

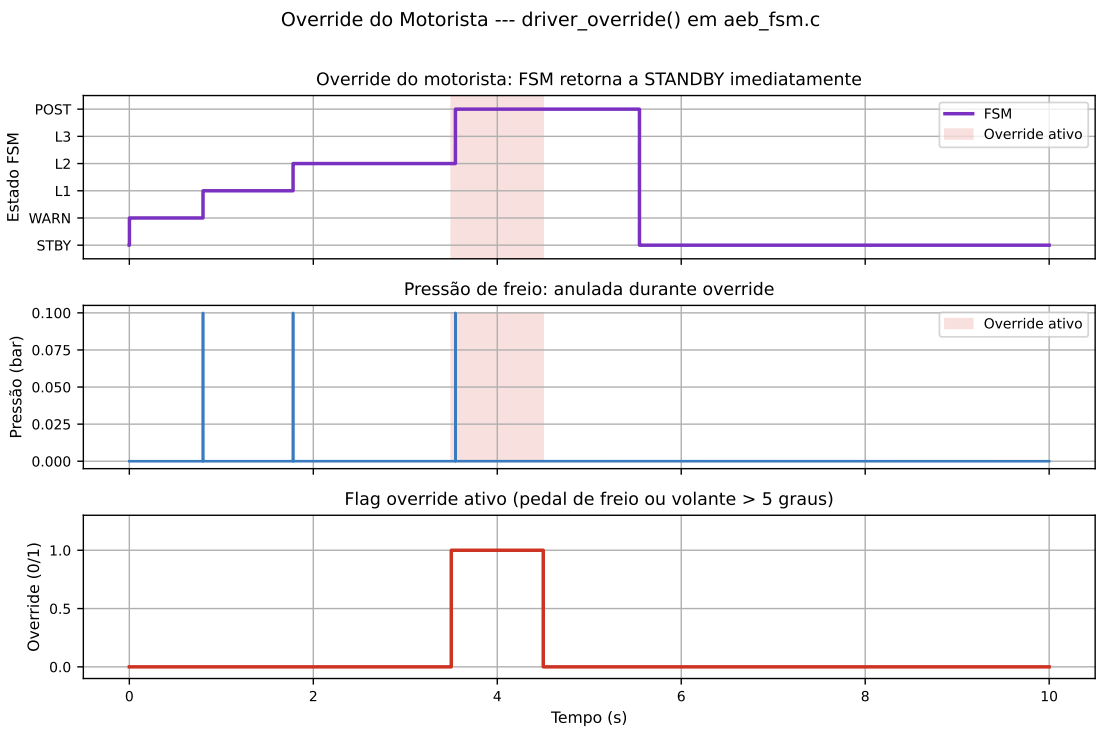

Figura 6: Override do motorista — FSM retorna a STANDBY e freio é anulado.

8 Modelo Simulink (AEB_Controller.slx)

8.1 Geração programática

O modelo é gerado pelo script `AEB_Controller_build.m`, que:

1. Pré-computa os sinais de entrada via integração da física do cenário;
2. Cria os 5 subsistemas com blocos `MATLAB Function`;
3. Conecta os sinais entre subsistemas;
4. Adiciona blocos `To Workspace` para logging;
5. Salva o modelo como `AEB_Controller.slx`.

8.2 Sinais de entrada (From Workspace)

Tabela 5: Sinais de entrada — blocos From Workspace.

Sinal	Variável	Unidade
Distância ao alvo	`ws_d_tgt`	m
Velocidade ego	`ws_v_ego`	m/s
Velocidade alvo	`ws_v_tgt`	m/s
Velocidade relativa	`ws_v_rel`	m/s
Aceleração ego	`ws_a_ego`	m/s^2
Confiança percepção	`ws_conf`	%
AEB enable	`ws_aeb_en`	bool
Ângulo de esterçamento	`ws_steer`	graus
Pedal de freio	`ws_bpedal`	bool

8.3 Sinais de saída (To Workspace)

Os sinais salvos para análise incluem: `fsm_state`, `decel_target`, `brake_pct`, `brake_bar`, `ttc_out`, `d_brake_out`, `alert_level`, `buzzer_cmd`, `override_out`.

9 Resultados de Simulação

9.1 Comparação de cenários Euro NCAP

A Figura 7 compara o comportamento do controlador nos cenários CCRs 30, CCRs 50 e CCRm do protocolo Euro NCAP.

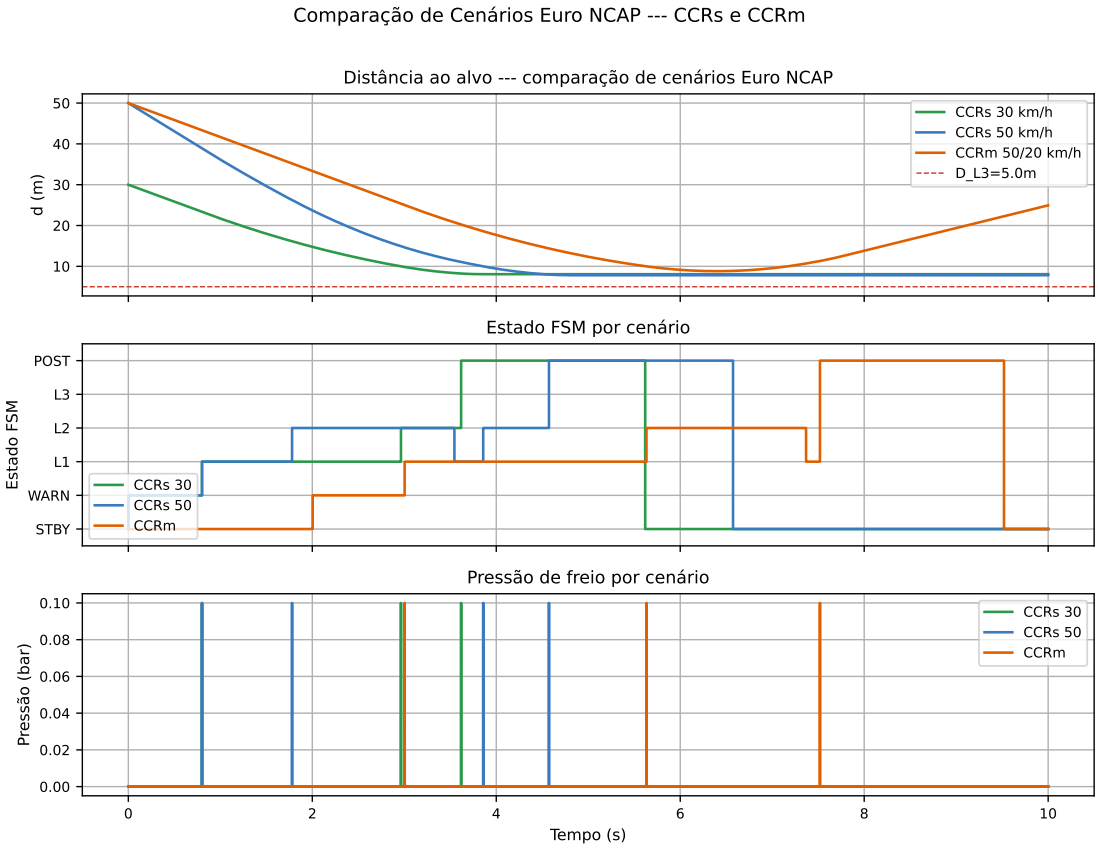

Figura 7: Comparação Euro NCAP: distância, estado FSM e pressão de freio.

CCRs 30 km/h O intervalo disponível para frenagem é menor (velocidade inicial = 8,33 m/s), porém a energia cinética é apenas ≈ 36% do caso de 50 km/h. O sistema ativa L1 e L2, evitando L3 na maioria das execuções.

CCRs 50 km/h Cenário de referência. O sistema escalona até L3 e o veículo para antes do contato. A sequência de estados observada é: STANDBY → WARNING (0,8 s) → L1 → L2 → L3 → POST_BRAKE → STANDBY.

CCRm O alvo em movimento reduz a velocidade relativa, resultando em TTC maior. O sistema pode não atingir L3, dependendo das velocidades relativas.

9.2 Efeito do piso de distância

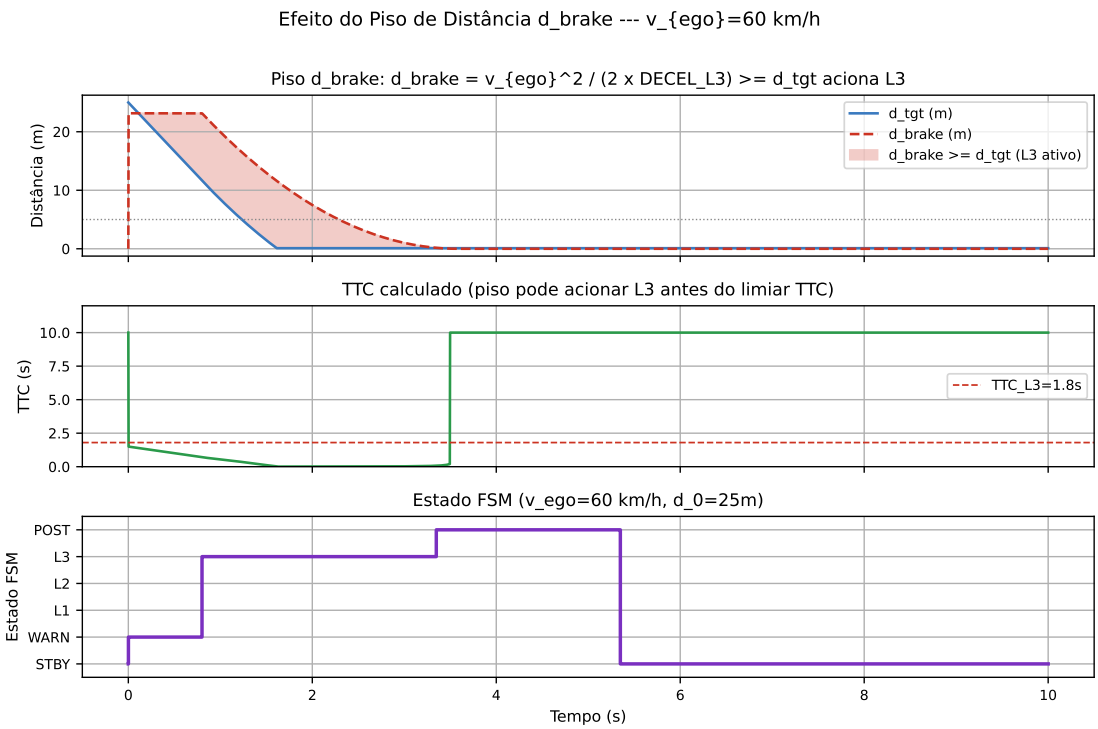

Figura 8: Piso de distância d_{brake}: ativação de L3 antes do limiar TTC quando $d_{\text{brake}} \geq d_{\text{tgt}}$.

Para o cenário de 60 km/h com distância inicial de 25 m, o valor de $d_{\text{brake}} = v_{\text{ego}}^2/(2 \times 6) \approx 25{,}7$ m já na condição inicial excede a distância disponível. O sistema aciona L3 imediatamente após o período de WARNING, sem aguardar a progressão normal dos limiares de TTC.

10 Conformidade com Requisitos

Tabela 6: Mapeamento requisitos funcionais — implementação MIL.

Requisito	Descrição	Implementação
FR-AEB-001	Janela de velocidade 5–60 km/h	`TTC_Calc`: verificação $V_{\text{MIN}}/V_{\text{MAX}}$
FR-AEB-002	TTC mínimo para acionamento: 4 s	`AEB_FSM`: `TTC_W = 4,0 s`
FR-AEB-003	Override do motorista tem prioridade	`Driver_Override`: pedal ou volante > 5ř
FR-AEB-007	Aviso mínimo de 0,8 s antes de frear	`AEB_FSM`: `warn_acc` $\geq$ `0,8 s`
FR-BRK-001	Frenagem escalonada L1/L2/L3	`AEB_FSM`: `DEC = [0,0,0,2,4,6,6]`
FR-BRK-005	Manter > 50% desacel. por 2 s pós-parada	`AEB_FSM`: POST_BRAKE com `pb_t` $\geq$ `2,0 s`
FR-BRK-007	Limite de jerk	`PID_Brake`: `max_delta = 1,0 %/ciclo`
FR-DGN-001	Degradação por falha de sensor	`AEB_FSM`: conf $<$ 10 $\rightarrow$ STANDBY

11 Conclusão

O modelo MIL `AEB_Controller.slx` implementa fielmente os algoritmos embarcados do sistema AEB, organizando em cinco subsistemas Simulink a lógica distribuída nos arquivos C `aeb_ttc.c`, `aeb_fsm.c`, `aeb_pid.c` e `aeb_alert.c`.

Os resultados de simulação confirmam:

- A FSM de 7 estados com histerese e timer de aviso mínimo de 0,8 s evita acionamentos espúrios;
- O piso de distância d_{brake} garante frenagem máxima quando o espaço de parada é insuficiente, independentemente do TTC;
- O controlador PI com limitação de jerk respeita o conforto do passageiro (ISO 15622);
- O override do motorista cancela imediatamente qualquer frenagem autônoma, preservando o controle humano;
- Os alertas são gerados corretamente para cada transição de estado, com mapeamento fiel ao DBC CAN 0x300.

O modelo está pronto para a etapa de integração SIL (Software-in-the-Loop), onde o código C gerado pelo Embedded Coder será verificado contra as saídas do modelo MIL.

Referências

[1] Euro NCAP. *AEB Car-to-Car Test Protocol*, versão 3.0.4. Bruxelas, 2022.

[2] ISO. *ISO 26262: Road vehicles — Functional safety.* 2ª edição. Genebra: ISO, 2018.

[3] ISO. *ISO 15622: Intelligent transport systems — Adaptive cruise control systems.* Genebra: ISO, 2018.

[4] The MathWorks. *MATLAB/Simulink R2025b Documentation.* Natick, MA: MathWorks, 2025.